

Trees

Data Structures

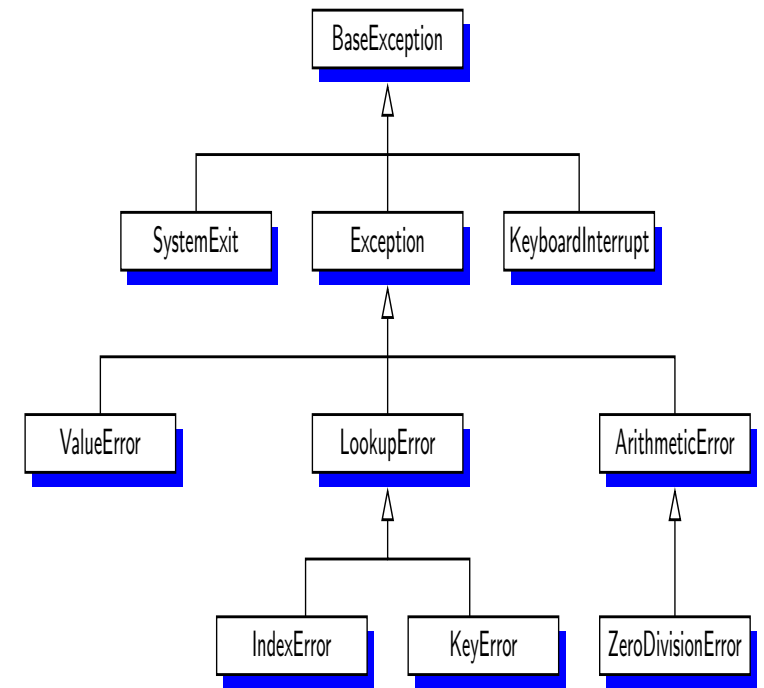
Chapter 8 in textbook

Based (in part) on textbook slides



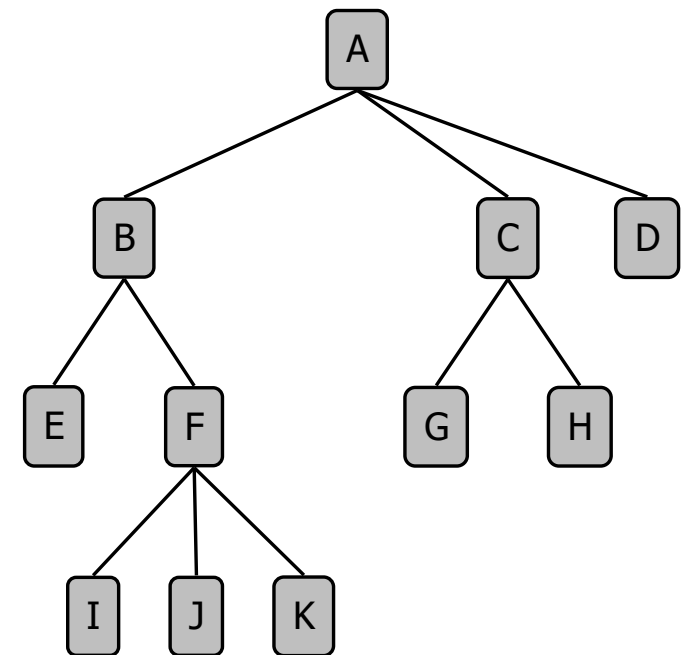
Trees

- **Trees** are one of the most important data-structure in whole of computer science.
- Allow us to organize data in an *hierarchical* fashion
 - (Unlike linear data-structures like arrays and lists)
 - Tree is a natural organization for e.g.:
 - File systems
 - Web-sites
 - Graphical User Interfaces (GUI's)
 - Databases (e.g., indices)
 - Inheritance in OO-Programming
 - ...



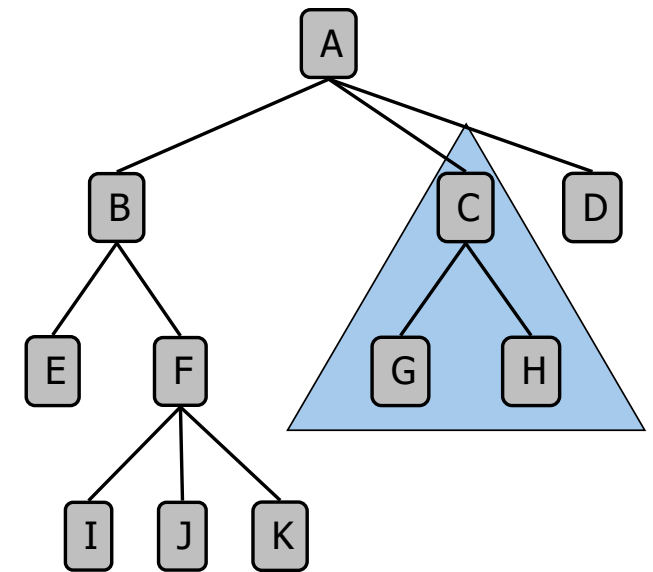
Tree Definition

- Formally, we define a **tree** T as a set of **nodes** storing elements such that the nodes have a **parent-child** relationship that satisfies the following properties:
 - If T is *non-empty*, it has a special node, called the **root** of T , that has no parent.
 - Each node v of T other than the root has a unique **parent** node w .
 - Every node with parent w is a **child** of w .
- Note that the definition allows for an empty tree, i.e. with no nodes! (makes recursive definitions easier).



Tree Terminology

- **root:**
 - A node without a parent (A)
- **internal node:**
 - node with at least one child (A, B, C, F)
- **external node** (also called a **leaf**):
 - node without children (E, I, J, K, G, H, D)
- **ancestors** of a node:
 - parent, grandparent, grand-grandparent, etc. (e.g., B and A are ancestors of F)
- **descendant** of a node:
 - child, grandchild, grand-grandchild, etc. (e.g., E, F, I, J, K are descendant of B)
- **depth** of a *node*:
 - number of ancestors the nodes has
- **height** of a *tree*:
 - maximum depth of any node (3 in the example to the right)
- **subtree**:
 - A tree consisting of a *node* and all its *descendants*



Tree Abstract Data Type (ADT)

- We use ***positions*** to abstract node accesses (as we did for *positional lists*)
- A ***tree ADT*** typically includes methods like:

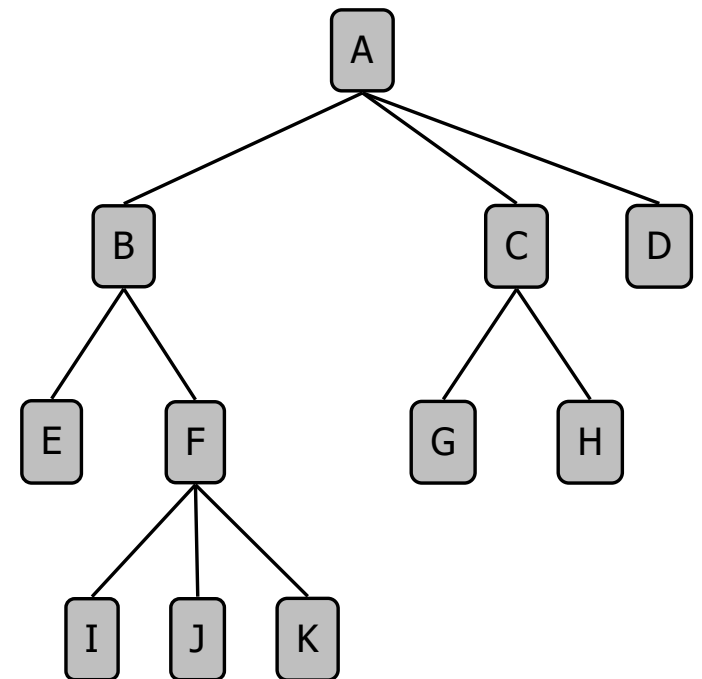
- **size()** -> int
- **is_empty()** -> bool
- **str()** -> str
- **height()** -> int
- **root()** -> Position
- **parent**(pos: Position) -> Position
- **children**(pos: Position) -> list[Position]
- **is_leaf**(pos: Position) -> bool
- **is_root**(pos: Position) -> bool
- **get_at** (pos: Position) -> object

- **clear()**
- **replace**(pos: Position, elem: object) -> object
- **add_root**(elem: object) -> Position
- **add_child**(pos: Position, elem: object) -> Position
- **remove_subtree**(pos: Position)
- **remove**(pos: Position)

Question: What would you think is a reasonable behaviour for **remove** when trying to remove an internal node? Why?

Tree Traversal

- We could *traverse* linear data-structures like lists from *front-to-back* or *back-to-front*.
- How about *traversing* trees?
 - Start at the *root*, then visit the *children* in *depth-first* (or possibly *breadth-first*) fashion
 - Assuming a *depth-first* traversal, when to “process” (e.g., print) the element?
 - **Pre-order**
 - **Post-order**
 - **In-order** (for binary trees)



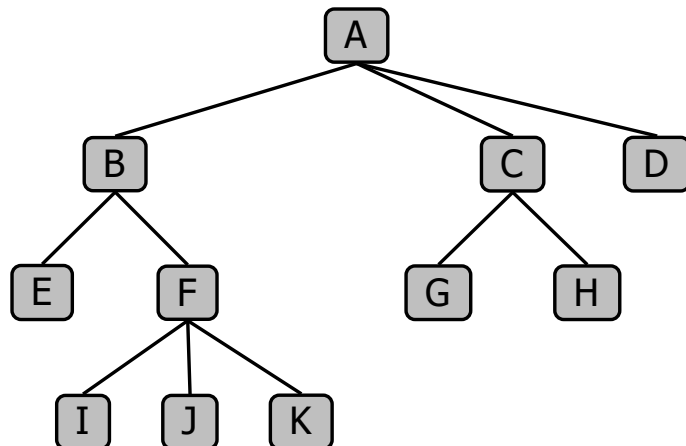
Tree Traversal

- **Pre-order**

- A, B, E, F, I, J, K, C, G, H, D

- **Post-order**

- E, I, J, K, F, B, G, H, C, D, A



Algorithm Tree traversal

```
1: function PRE_ORDER(T, pos)
2:   if pos is NIL then
3:     return
4:   end if
5:   VISIT(T.get_at(pos))  ▷ Process node before its children
6:   for all child in T.children(pos) do
7:     PRE_ORDER(T, child)
8:   end for
9: end function

10:
11: function POST_ORDER(T, pos)
12:   if pos is NIL then
13:     return
14:   end if
15:   for all child in T.children(pos) do
16:     POST_ORDER(T, child)
17:   end for
18:   VISIT(T.get_at(pos))  ▷ Process node after its children
19: end function
```

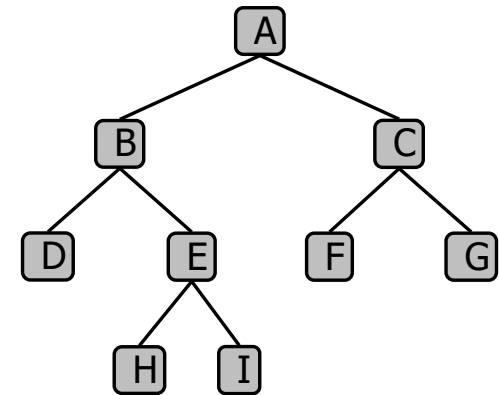
Question: Which code represents the **base case(s)** for these recursive functions? (indicate the line numbers)

Binary Trees

- A **binary tree** is a tree with the following properties:
 - Each *internal* node has **at most two children** (exactly two for *proper / strict* binary trees)
- We call the children **left child** and **right child**
 - **left_child**(pos: Position) -> Position
 - **right_child**(pos: Position) -> Position

Algorithm Tree traversal

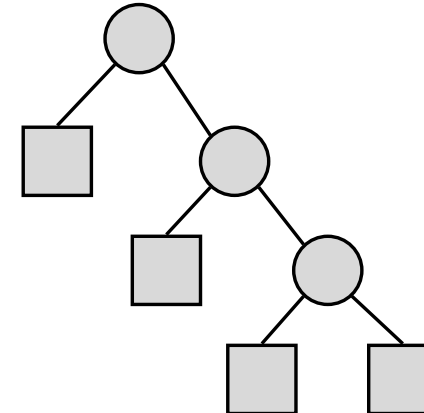
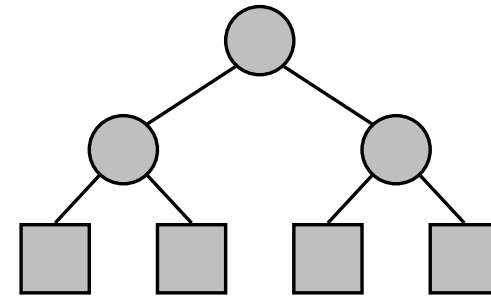
```
1: function IN_ORDER(T, pos)  ▷ Only for binary trees
2:   if pos is NIL then
3:     return
4:   end if
5:   IN_ORDER(T, T.left_child(pos))
6:   VISIT(T.get_at(pos))  ▷ ... in between its two children
7:   IN_ORDER(T, T.right_child(pos))
8: end function
```



- **In-order**
D, B, H, E, I, A, F, C, G

Properties of Proper Binary Trees

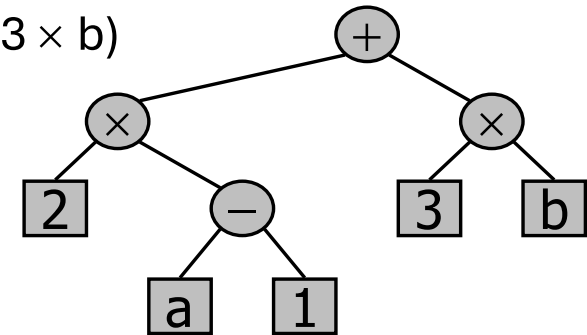
- Notation
 - n number of nodes
 - e number of external nodes (shown as $\square$)
 - i number of internal nodes (shown as $\circ$)
 - h height
- Properties
 - $e = i + 1$
 - $n = 2e - 1$
 - $h \leq i$
 - $h \leq (n - 1)/2$
 - $e \leq 2^h$
 - $h \geq \log_2 e$
 - $h \geq \log_2 (n + 1) - 1$



Tree Example Usage: *Expression Trees*

- Compilers/Interpreters must know how to represent arithmetic expressions correctly to evaluate them
 - You will learn more about later in the course *Programming Languages* (and elective course *Compilers!*)
- They use an **expression tree** internally to represent each expression.
 - An expression tree is a **binary tree** associated with an arithmetic expression
 - internal nodes: operators
 - external nodes: operands

$$2 \times (a - 1) + (3 \times b)$$

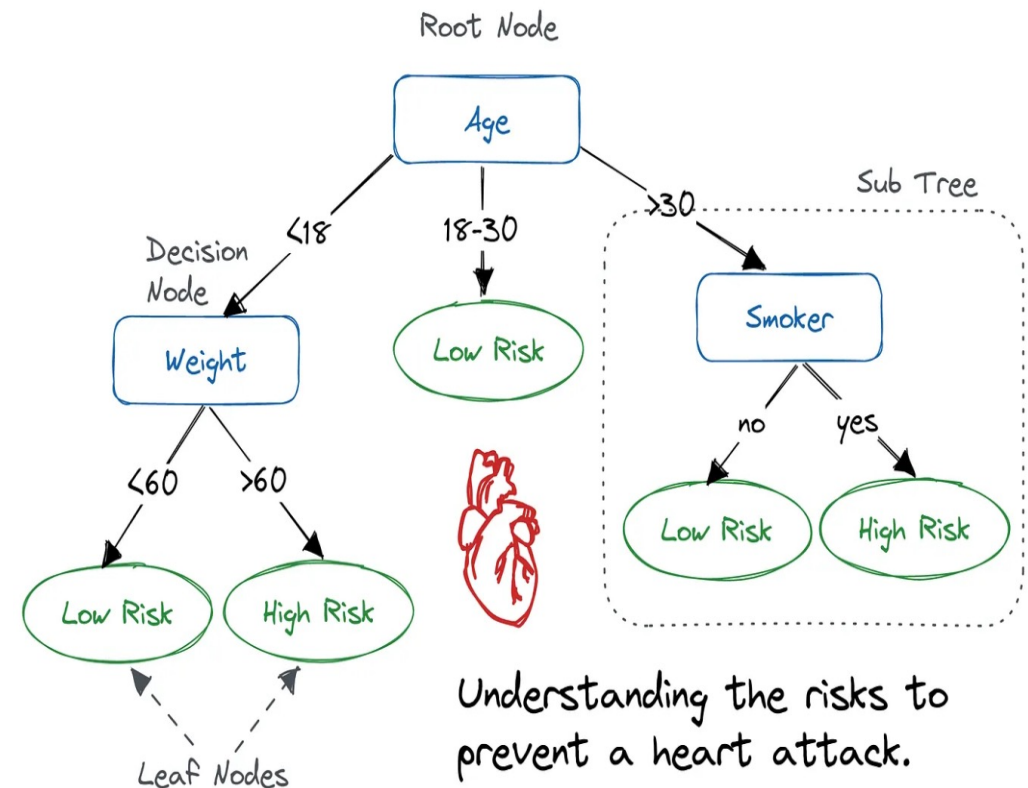


Algorithm Evaluate

```
1: function EVALUATE(T, pos)
2:   if T.is_leaf(pos) then
3:     v ← T.get_at(pos)
4:   else
5:     l ← EVALUATE(T, T.left_child(pos))
6:     r ← EVALUATE(T, T.right_child(pos))
7:     v ← compute(T.get_at(pos), l, r)
8:   end if
9:   return v
10: end function
```

Tree Example Usage: *Decision Trees*

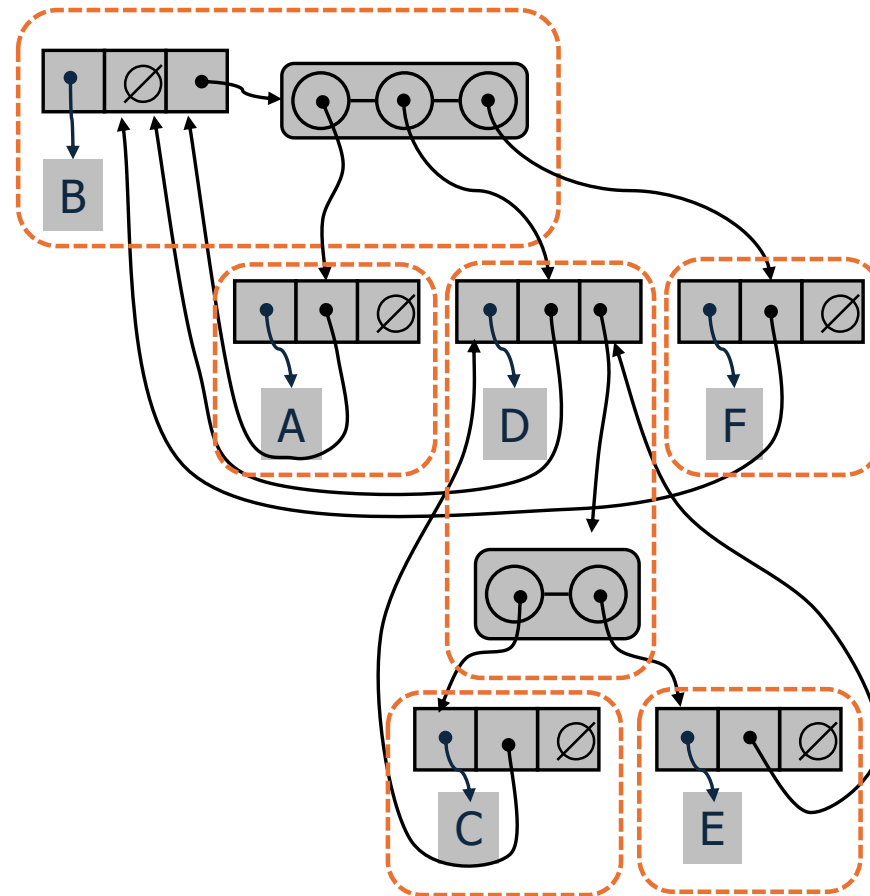
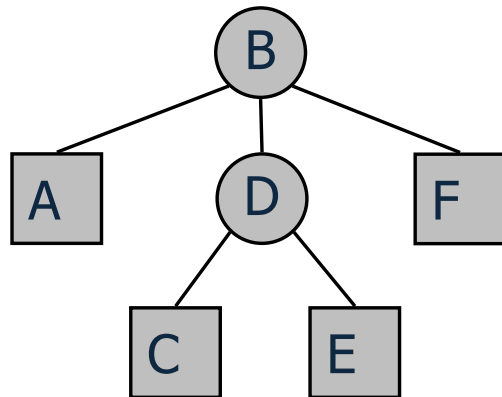
- In *data-science* we often like to build *predictive models* from data
 - You will learn more about that later if you take a course in *artificial intelligence*, *machine learning*, or *data-science*.
- One such model is a **decision tree**
 - Internal nodes represent questions.
 - External nodes represent answers (decisions).



Source: <https://www.datacamp.com/tutorial/decision-tree-classification-python>

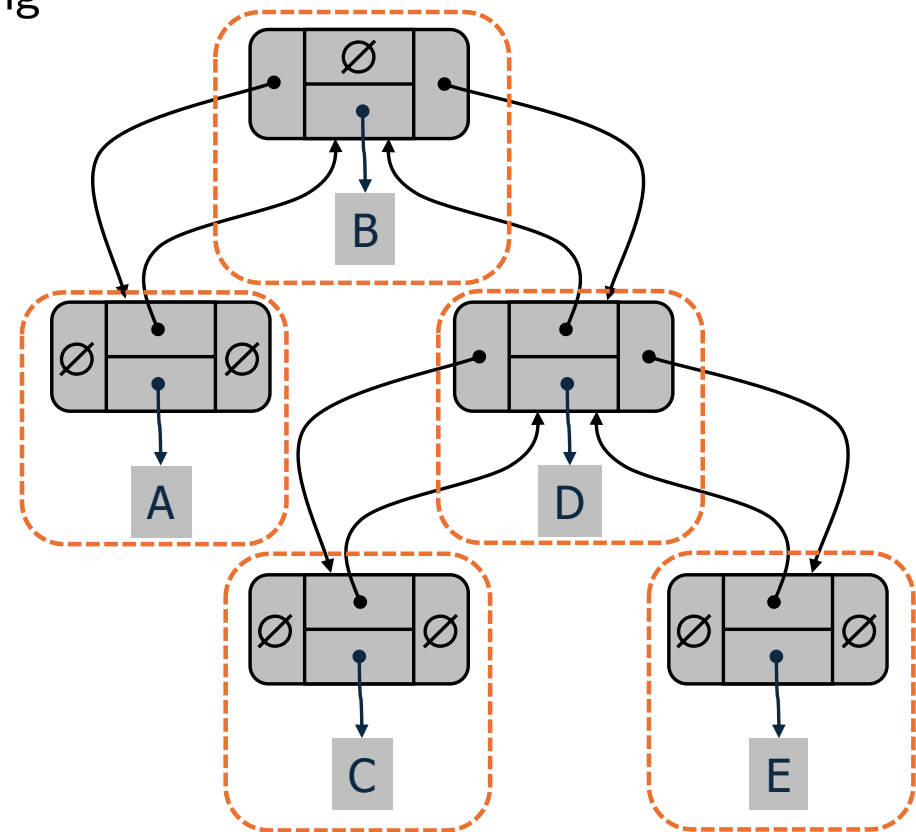
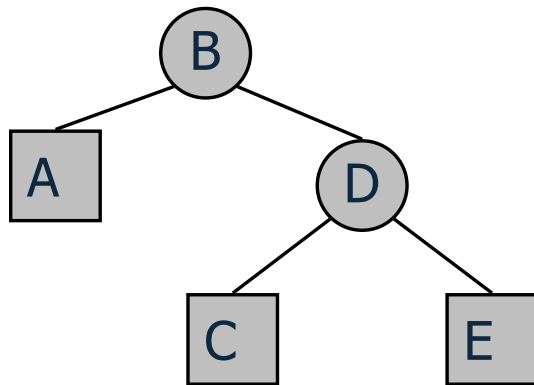
Implementing **Tree ADT** (using a linked structure)

- A *node* is represented by an object storing (references to)
 - element
 - parent node
 - sequence of children nodes



Implementing **Binary Tree ADT** (using a *linked* structure)

- A node is represented by an object storing (references to):
 - element
 - parent node
 - left child node
 - right child node



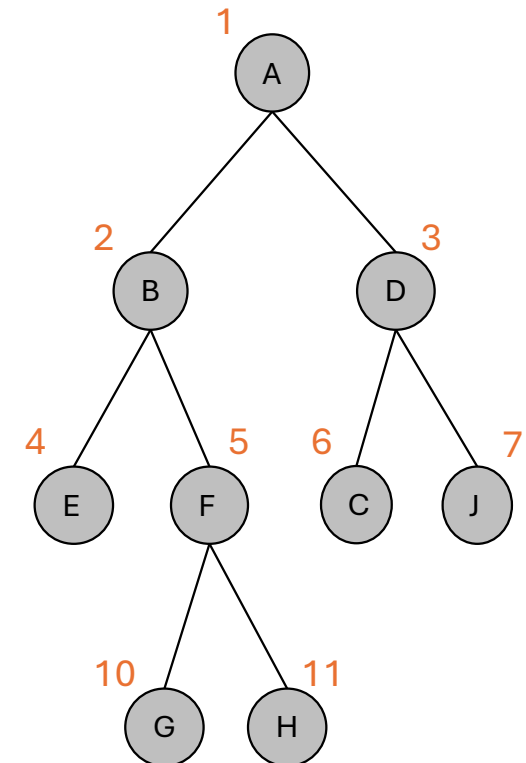
Implementing **Binary Tree ADT** (using an *array*)

- Nodes are stored in an array A



Node v is stored at $A[\text{rank}(v)]$

- $\text{rank}(\text{root}) = 1$
- if node is the left child of $\text{parent}(\text{node})$,
 - $\text{rank}(\text{node}) = 2 \cdot \text{rank}(\text{parent}(\text{node}))$
- if node is the right child of $\text{parent}(\text{node})$,
 - $\text{rank}(\text{node}) = 2 \cdot \text{rank}(\text{parent}(\text{node})) + 1$



Summary

- A **tree** is one of the most fundamental abstract data type in all of computer science
 - Many algorithms rely on it for effective problem solving
 - A wide range of applications rely on it
- A collection of **nodes** organized in a hierarchical parent/child manner
 - internal / external
 - root / leaf
- ADT implemented using either:
 - **Linked** structure
 - **Array-based** structure (for binary tree)

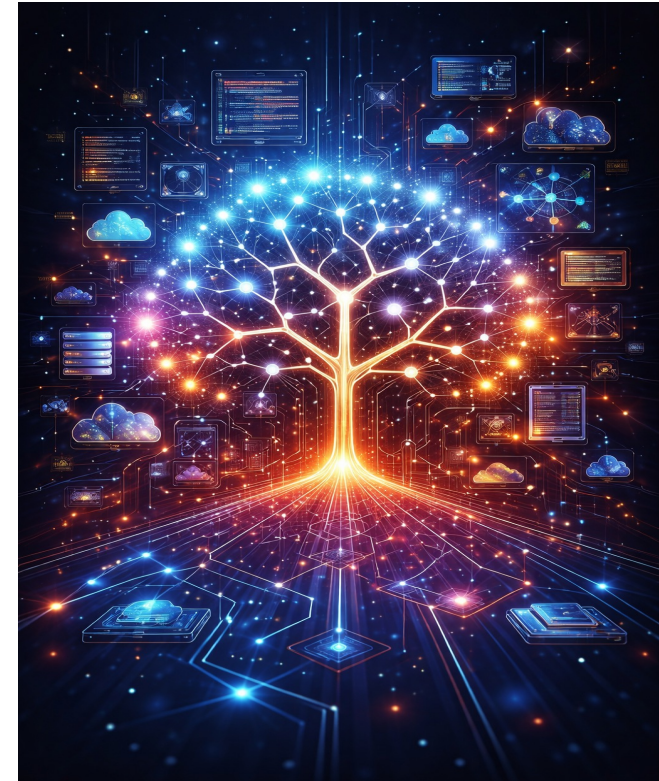


Image created using ChatGPT

Question: What do you think are the (relative) pros and cons of those two structures for implementing a **binary tree**?